



Universidad
Indoamérica

UNITAT 6 · FONAMENTS DE SISTEMES OPERATIUS

Senyals i memòria compartida

Universidad Tecnológica Indoamérica

Laboratori TD5

Senyals: notifikacions asíncrones a un procés

El nucli

SIGSEGV (accés no vàlid), SIGBUS, SIGPIPE (canonada sense lector), SIGCHLD (un fill ha acabat), SIGALRM

El terminal

Ctrl-C → SIGINT, Ctrl-\ → SIGQUIT, Ctrl-Z → SIGTSTP, tancar → SIGHUP

Un altre procés

`kill(pid, sig)` i l'ordre `kill`: SIGTERM, SIGKILL, SIGUSR1...

Canvien l'estat del procés: l'acaben, l'aturen, el reprenen... o n'executen una funció.

Enviament asíncron, senyals pendents i quan es capturen

- L'enviament pot passar en qualsevol moment, tant si el destinatari s'executa com si no
- **Pendent**: rebut però encara no lliurat (bloquejat, o el nucli encara no l'ha comprovat)
- Es lliura en **tornar al mode usuari**: després d'una crida al sistema o d'un canvi de context

Aclariment a l'original («no existeix cap cua de senyals»): és cert per als senyals *estàndard* —un de pendent és un bit, les repeticions es perden—, però els senyals de **temps real** (`SIGRTMIN` ... `SIGRTMAX`) sí que fan cua i poden portar un valor (`sigqueue`).

kill i els senyals més habituals

C

```
int kill(pid_t pid, int sig);
/* pid > 0: that process · 0: my process group
   < -1: group -pid · -1: every process I may signal
   errors: EINVAL, EPERM, ESRCH */
```

Correcció a l'original (diapositiva 97): deia que SIGHUP, SIGINT, SIGTSTP... són «no emmascarables i no bloquejables». Només **SIGKILL** i **SIGSTOP** no es poden capturar, ignorar ni bloquejar; tots els altres sí.

SENYAL	PER DEFECTE
SIGHUP · SIGINT · SIGTERM	acabar
SIGQUIT · SIGSEGV · SIGBUS	acabar + core
SIGKILL	acabar (sempre)
SIGPIPE · SIGALRM · SIGUSR1 · SIGUSR2	acabar
SIGSTOP · SIGTSTP	aturar
SIGCONT	continuar
SIGCHLD	ignorar

Pendents, bloquejats i lliurats

senyal	n	pendent	bloquejat	acció	per defecte
SIGINT (Ctrl-C)	2	0	—	SIG_DFL	acabar
SIGQUIT (Ctrl-\)	3	0	—	SIG_DFL	acabar + core
SIGKILL	9	0	impossible	SIG_DFL	acabar
SIGUSR1	10	1	sí	gestor	acabar
SIGSEGV	11	0	—	SIG_DFL	acabar + core
SIGPIPE	13	0	—	SIG_DFL	acabar
SIGALRM	14	0	—	SIG_DFL	acabar
SIGCHLD	17	0	—	SIG_DFL	ignorar
SIGCONT	18	0	—	SIG_DFL	continuar
SIGSTOP	19	0	impossible	SIG_DFL	aturar
SIGTSTP (Ctrl-Z)	20	0	—	SIG_DFL	aturar
SIGRTMIN	34	0	—	SIG_DFL	acabar

procés

adormit a pause / sigsuspend

què ha passat

- sigprocmask(SIG_BLOCK, {SIGUSR1}); while (!flag)...
- kill(pid, SIGUSR1): el nucli marca SIGUSR1 com a pendent
- arriba en el pitjor moment, però queda PENDENT (bloquejat)
- sigsuspend(màscara sense SIGUSR1): desbloqueja i espera en UNA sola operació

Gestors: de `signal` a `sigaction`

C

```

sighandler_t signal(int sig, sighandler_t f); /* obsolete; SIG_ERR on error */
int sigaction(int sig, const struct sigaction *act,
              struct sigaction *old);
struct sigaction {
    void (*sa_handler)(int); /* or SIG_DFL, SIG_IGN */
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask; /* blocked during the handler */
    int sa_flags; /* SA_RESTART SA_SIGINFO SA_NOCLDSTOP... */
};

```

- Durant el gestor, el mateix senyal queda bloquejat
- En tornar, el programa continua on era. `sa_flags`: `SA_RESTART` torna a començar la crida interrompuda (si no, `EINTR`, unitat 2); `SA_SIGINFO` fa servir `sa_sigaction`; `SA_NOCLDSTOP`: `SIGCHLD` només quan un fill acaba, no quan s'atura o continua
- A dins, només funcions **segures davant senyals asíncrons** (`write` sí, `printf` i `malloc` no) i variables `volatile sig_atomic_t`
- `signal()` té semàntiques diferents segons el sistema: fes servir `sigaction`

Esperar i programar: `pause`, `alarm`, `sleep`

- `pause()` : dorm fins que un senyal l'acaba o executa un gestor; sempre retorna -1
- `alarm(s)` : SIGALRM d'aquí a `s` segons (per defecte, acaba el procés)
- `sleep(s)` : suspèn durant `s` segons

Actualització (diapositiva 101): l'original deia que `sleep` s'implementa amb `pause` + `alarm` i que no s'han de barrejar. A glibc/Linux, `sleep` fa servir `clock_nanosleep` i no toca l'alarma. Avui es prefereixen `nanosleep` i els temporitzadors `timer_create` o `timerfd`.

Els exemples de l'original... i el seu error amagat

c

```
/* slide 102: mysleep */
signal(SIGALRM, do_nothing);
alarm(s);
pause();      /* what if SIGALRM arrives BEFORE pause? */

/* slide 103: ping-pong, parent and child after fork() */
kill(getppid(), SIGUSR1);
pause();      /* same race: the answer may come first */
```

- Si el senyal arriba **entre** la preparació i `pause()`, el gestor s'executa abans d'hora i `pause` espera per sempre (diapositiva 105)
- Un error difícil de detectar i reproduir: depèn del planificador (com la cursa de `mypipe`, unitat 5)
- La conclusió de l'original (diapositiva 104): **no facis servir** `signal()` / `pause()` per sincronitzar

Bloquejar, comprovar i sigsuspend

C tds/td5/mysignal.c

```
/* Block SIGUSR1 outside sigsuspend: a signal arriving
   and the wait stays pending instead of being lost (th
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1);
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unbl
```

sigprocmask bloqueja; sigsuspend

desbloqueja i dorm en una sola operació

atòmica. Operacions de conjunts: sigemptyset, sigfillset, sigaddset, sigdelset, sigismember.

Sortida real

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep
0.2; kill -STOP $P; kill -USR1 $P; sleep 0.2; kill -CONT
$P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status:
$? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1
./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

Senyal + memòria: l'emissor i el receptor

Sortida real

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

- El receptor escriu el seu PID al començament d'un fitxer mapat i espera SIGUSR1 amb `sigsuspend`
- L'emissor llegeix el PID, escriu el missatge al mateix fitxer mapat i envia el senyal: el senyal **avisa**, la memòria **porta**

Avui també: `signalfd` (llegir senyals com a dades des d'un descriptor), `pidfd_send_signal` (enviar senyals sense curses de PID reutilitzats).

Compartir memòria entre processos

- La memòria virtual aïlla: cada procés té les seves pàgines (unitat 4)
- Compartir = fer que pàgines virtuals de **dos processos** apuntin al **mateix marc físic**
- Cal un **punt de trobada**: un fitxer o un nom

mmap + fitxer

el fitxer és el punt de trobada i guarda les dades (persisteix)

shm_open +
mmap

POSIX, un nom a /dev/shm (a la RAM), recomanat avui

ftok +
shmget +
shmat

System V, clau numèrica; el contingut del fitxer no es fa servir

Mapar un fitxer a la memòria: `mmap`

C

```
void *mmap(void *addr, size_t len, int prot, int flags,  
           int fd, off_t offset); /* MAP_FAILED on error */  
int munmap(void *addr, size_t len);  
int msync(void *addr, size_t len, int flags);
```

- `prot`: `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`
- `flags`: `MAP_SHARED`, `MAP_PRIVATE` (còpia en escriptura), `MAP_ANONYMOUS`
- Errors: `EACCES` (no és un fitxer regular, o el mode del descriptor no permet el que es demana: `O_WRONLY` i `O_APPEND` no funcionen), `ENOMEM`

Correcció a l'original (diapositiva 118):

`len` no ha de ser múltiple de la mida de pàgina (s'arrodoneix cap amunt); el que ho ha de ser és `offset`. I accedir més enllà del final del fitxer no dona `SIGSEGV` sinó **SIGBUS**: la propera animació ho mostra amb la sortida real.

Compartir memòria a través d'un fitxer

C tds/td5/sender.c

```
int fd = open(FILENAME, O_RDWR);
exit_if(fd == -1, "open " FILENAME);
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");

...

shared[off] = '\\0';

exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also push to the file */
exit_if(kill(pid, SIGUSR1) == -1, "kill");
exit_if(munmap(shared, SIZE) == -1, "munmap");
```

- `O_RDWR`: `PROT_WRITE` + `MAP_SHARED` necessita un descriptor de lectura i escriptura
- Després de `mmap` es pot tancar el descriptor: el mapatge continua viu
- Escriure `shared[i]` és escriure el fitxer: el receptor, que mapa el mateix fitxer, ho veu
- `munmap` elimina el mapatge; `msync` força la còpia al disc

Els límits de `mmap`: `SIGBUS` i `PROT_*`

pàgina 0 (0 ... 4095)

zona llegible/escrivible més enllà del fitxer: es llegeix com a 0, no es desa

fitxer: 128 B

bytes escrits amb `map[i] = 'x'`

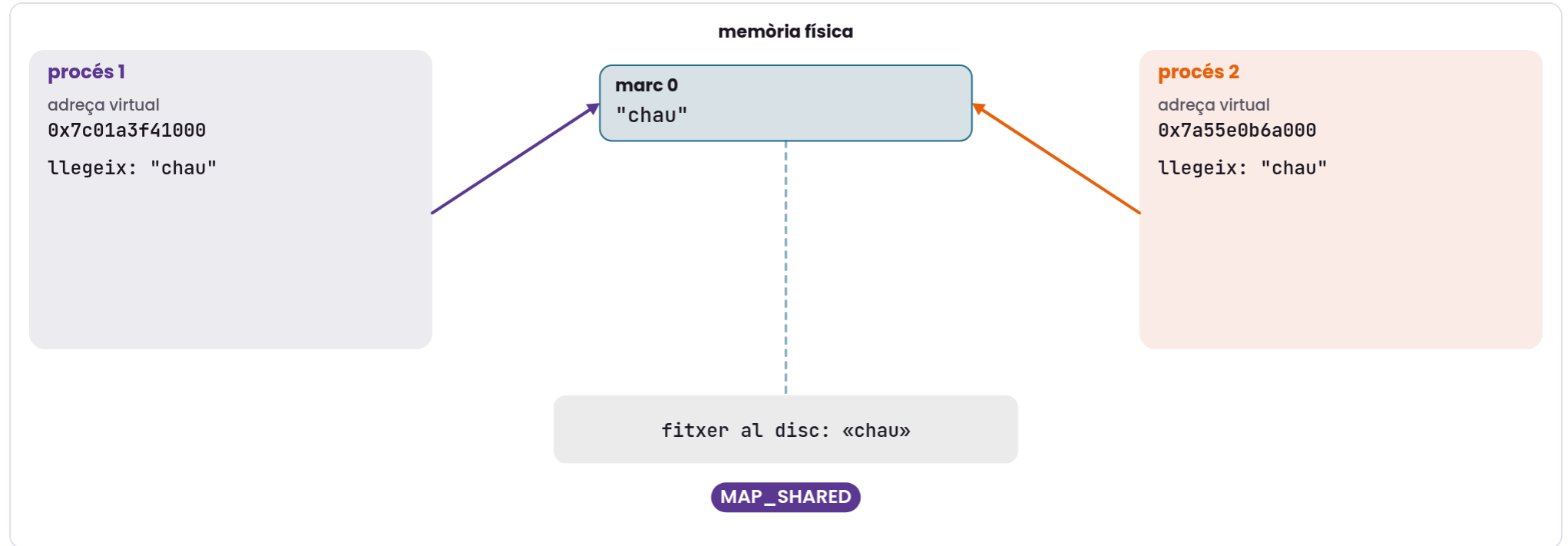
pàgina 1 (4096 ... 8191)

sense fitxer al darrere → `SIGBUS`

`SIGBUS @ 4096`

mida del fitxer després: 128 B

Dos processos, un marc físic



shm de System V: `ftok` → `shmget` → `shmat`

C tds/td5/shm_counter.c

```

key_t key = ftok(KEYFILE, KEYID);
exit_if(key == -1, "ftok");
int id = shmget(key, SIZE, IPC_CREAT | 0644);
exit_if(id == -1, "shmget");

if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl I");
    printf("segment %d removed\n", id);
    return EXIT_SUCCESS;
}

int *counter = shmat(id, NULL, 0);
exit_if(counter == (void *)-1, "shmat");
counter[0]++; /* a fresh
printf("segment %d (key 0x%08x): counter = %d\n", id, (
exit_if(shmdt(counter) == -1, "shmdt");

```

Sortida real

```

$ ./shm_counter; ./shm_counter; ./shm_counter;
./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed

```

- El segment sobreviu als processos fins a `shmctl(IPC_RMID)`; `ipcs -m` els llista
- Cada procés el veu en una altra adreça: **no hi guardis mai punters**
- `IPC_PRIVATE` : sense clau · `SHM_RDONLY` : només lectura · `shmctl` : `IPC_STAT` llegeix, `IPC_SET` canvia permisos, `IPC_RMID` destrueix

Compartir memòria exigeix sincronitzar

Sense operació atòmica

```
( slot = seg[0] + 1; seg[0] = slot; ):
```

Sortida real

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Amb `__atomic_add_fetch`:

Sortida real

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566
357567 357572 357568 357571 357569 357565 357574 357573
357570 357576 357578 357575 357577 357579
```

És el tema de la unitat 7: condicions de cursa, mutex, semàfors.

TD5 resol't + repte IA

- `mysignal` amb `sigsuspend`; l'emissor i el receptor
- `mmap_limits` i les cinc proteccions
- `shm_counter`, `shm_fork` i la cursa `shm_race`

Bash

```
make -C tds && tds/tests/run.sh td5
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. Per què tres SIGUSR1 bloquejats només executen el gestor una vegada?
2. Quin problema resol `sigsuspend` que `pause` no resol?
3. Per què no es guarden punters a la memòria compartida?